🤖 AI-Native HR Agent

System Architecture & Design

Team Kanyaraasi | Track 2 - AI HR Agent | AWS Hackathon 2026

🖥️ Presentation Layer

Streamlit UI (app_ai_native.py)

Interactive web interface with 6 functional tabs

- Resume Screening Dashboard
- Interview Scheduling Interface
- Questionnaire Generator
- Pipeline Management View
- Leave Management Portal
- Escalation Handler

AI Insights Display

Executive summary and strategic insights visualization

- Executive Summary Cards
- Cross-Candidate Insights
- Predictive Recommendations
- Confidence Scoring

⚙️ Application Layer

AI-Native Agent (hr_agent_ai_native.py)

Enhanced agent with AI-powered features

- Executive Summary Generation
- Strategic Insights Engine
- Predictive Recommendations
- Percentile Rankings

Base HR Agent (hr_agent_upgraded.py)

Core agent with 6 functional modules

- Resume Screener
- Interview Scheduler
- Question Generator
- Pipeline Validator
- Leave Manager
- Query Escalator

LLM Manager (gemini_llm_manager_upgraded.py)

AI integration with fallback mechanisms

- Gemini 2.5 Flash Integration
- Template Fallback System
- Prompt Engineering
- Response Parsing

🧠 Business Logic Layer

Ranking Engine

Deterministic candidate scoring

- Skills Match (60%)
- Experience Weight (20%)
- Preferred Skills (10%)
- Resume Quality (10%)
- Optional LLM Boost (15%)

Scheduling Logic

Constraint-safe slot allocation

- Slot Validation
- Conflict Detection
- Deterministic Assignment
- Pipeline State Update

State Machine

Pipeline state management

- Transition Validation
- Terminal State Handling
- Audit Trail Logging
- Status Tracking

Policy Enforcement

Leave management rules

- Balance Validation
- Quota Enforcement
- Notice Period Check
- Consecutive Days Limit

💾 Data Layer

Data Loader (data_loader.py)

Data ingestion and transformation

- CSV Parsing
- Candidate Object Creation
- Job Description Builder
- Skill Extraction

Data Sources

Input datasets

- resume_dataset_1200.csv
- employee leave tracking data.xlsx
- Interview Slots (generated)
- Leave Policies (configured)

Export Interface

Standardized JSON output

- Pipeline Data
- Interview Schedule
- Leave Decisions
- Escalation Log
- Transition History

🌐 External Services

Gemini 2.5 Flash API

Google's LLM for semantic understanding

- Resume-Job Matching
- Question Generation
- Insight Synthesis
- Recommendation Logic

Fallback System

Template-based generation

- Rule-based Matching
- Deterministic Logic
- Fast Mode (2-5s)
- 100% Reliability

📊 Data Flow

1 **User Input**

User clicks "Run AI Analysis" → Streamlit UI captures configuration (LLM enabled, fast mode)

2 **Data Loading**

Data Loader reads CSV files → Creates 1200 Candidate objects → Builds Job Description

3 **Agent Initialization**

AI-Native Agent initialized → Leave balances created for all employees → LLM Manager configured

4 **Resume Screening**

Ranking Engine scores candidates → Optional LLM semantic boost → Sorted by match score

5 **AI Insights Generation**

Executive Summary created → Cross-candidate patterns identified → Predictive recommendations generated

6 **Results Display**

UI renders insights → Top candidates shown → All 6 modules accessible via tabs

7 **Export**

User downloads JSON → Standardized format → All data included (pipeline, schedule, leave, escalations)

🎯 Six Core Modules

1. Resume Screening

Purpose: Rank candidates by match score

- Deterministic scoring (60-10-20-10)
- Explainable ranking
- Reasoning chains
- Fast mode: 2-5s for 1200 candidates

2. Interview Scheduling

Purpose: Assign interview slots

- Constraint-safe allocation
- Zero conflicts guaranteed
- Slot validation
- Pipeline state update

3. Questionnaire Generation

Purpose: Create personalized questions

- Technical questions
- Behavioral (STAR format)
- Situational scenarios
- Role-specific focus

4. Pipeline Management

Purpose: Track candidate status

- State machine validation
- Transition logging
- Terminal state handling
- Complete audit trail

5. Leave Management

Purpose: Process leave requests

- Policy enforcement
- Balance tracking
- No underflow
- Clear denial reasons

6. Escalation Handling

Purpose: Categorize query priority

- Keyword-based classification
- High/Medium/Low priority
- Reasoning provided
- Recommended actions

🛠️ Technology Stack

Frontend

Streamlit 1.28+

Backend

Python 3.x

LLM

Gemini 2.5 Flash

Data Processing

Pandas 2.0+

API Client

google-generativeai

Configuration

python-dotenv

AI-Native HR Agent | Team Kanyaraasi | AWS Hackathon 2026
Architecture designed for scalability, modularity, and production readiness